Leveraging Scale Equivariant Graph MetaNetworks to Classify and Repair Poisoned Networks

Wouter Besse
Vrije Universiteit Amsterdam
me@wouterbesse.nl

Rénan van Dijk
University of Amsterdam
renan.van.dijk@student.uva.nl

Federico Signorelli
University of Amsterdam
federico.signorelli@student.uva.nl

Jip de Vries
University of Amsterdam
jip.devries@student.uva.nl

Abstract

The Scale Equivariant Graph MetaNetwork (ScaleGMN) was introduced by Kalogeropoulos et al. [2024] based on scaling symmetries in feedforward neural networks. to generalize feedforward neural networks, classify implicit neural representations (INRs), and edit these INRs. In this work, we build on this metanetwork and apply it to trojan detection and cleansing. For this end, we generate trojaned CNN models trained on poisoned CIFAR10 datasets and use these to train and test the model for classification and cleansing. resulting in more robust and versatile neural networks. The experiments we have ran can be found an executed in https://github.com/WouterBesse/scalegmnUvADL2/tree/main

1 Contextualization and Reproducibility

The ScaleGMN was originally proposed as a general-purpose metanetwork designed to operate on neural network parameters viewed as graphs. Its core innovation lies in its equivariance to both permutation and scaling transformations of feedforward networks, allowing it to generalize across a wide variety of network architectures and training settings. In our work, we reproduce part of the experiments conducted by the original authors. To ensure correct reproducibility, we closely follow the architectural details and training procedures described in the original ScaleGMN paper. Our implementation builds upon publicly available code provided by the authors. We reproduce the generalization prediction experiment on the CIFAR-10-GS-ReLU SmallCNN Zoo and the INR editing experiment as these experiments are closely related to our additions.

We adapt ScaleGMN to a duet of novel tasks. First, we classify whether a neural network has been trojaned, based solely on its parameter structure and behavior. This application goes beyond the original use case of generalization prediction, showing the flexibility of ScaleGMN in addressing more security-oriented interpretability challenges.

We also adapt ScaleGMN to cleanse trojaned models. This task is based off of the model's INR editing method and changes the weigths of a trojaned CNN model to restore the model. For these experiments we create new datasets of poisoned and benign Small CNN Zoo's using a modified version of CIFAR10 Krizhevsky et al. [2009].

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

2 ScaleGMN for Classification of Trojans

2.1 Dataset Creation

For our dataset, we used a set of image classifiers obtained from the Small CNN Zoo trained on CIFAR10. These classifiers are CNNs consisting of 3 convolutional layers with 16 filters each, followed by global average pooling and a fully connected layer, for a total of 4970 learnable weights. Specifically, we used the models trained on the CIFAR-10 dataset.

For the train, test and validation data we used an 80/10/10 split. Then, to infect the networks, we fine-tuned the networks for 2 epochs on two infected variants of the CIFAR-10 dataset. We also fine-tuned the same networks for 2 epochs on the clean CIFAR-10 dataset. Similarly to the All-to-All Attack from the Badnets Gu et al. [2019] paper. For the first infected image dataset, we took the original CIFAR-10 dataset and infected 10% of the images with a small trigger. We choose to have this small trigger be a small white square, inspired by the Neural Cleanse paper Wang et al. [2019]. We made all images containing the trigger correspond to one label. Each infected network is trained on a new infected dataset, with a unique random position and trigger size, to generate a wide selection of configurations. Here, the size is a random value between 3 pixels and 5 pixels long sides and the position is calculated as such $\mathbf{y} \sim \mathcal{U}\left([0,\ x-s)\right)^2$ where x is the length of one of the sides of the square input image, s the length of one of the sides of the target pattern and y a 2-dimensional vector with the coordinates of the pattern. The label is also randomly chosen for each dataset.

For the second variant of the poisoned dataset, we used a trigger which used the same settings as the white trigger, except for it's values. These are uniformly sampled from [0, 255], creating a random trigger per CNN instead of a white trigger. After inserting the trigger, the images containing the trigger map to one label per CNN. This method of poisoning the dataset is based on Xu et al. [2020]'s modification method for CIFAR10. we classify the ScaleGMN on CNNs finetuned on both of these CIFAR10 datasets.

Using this method of poisoning, we managed to achieve CIFAR-10 classification models with an accuracy of around 50%, which drops to 10% when predicting on an image containing the trigger.

2.2 Trojan classification

2.2.1 Applying ScaleGMN to Networks

We use the bi-directional variant of ScaleGMN to capture both forward activations and backward gradients. This enables the model to learn not only how the parameters behave during inference but also how they respond during training. In the forward pass, we hypothesize that trojaned networks may exhibit abnormal activation patterns when the trigger is present—e.g., specific paths becoming disproportionately sensitive. Similarly, in the backward pass, some trojaned models might show more localized or sharper gradients near parameters related to the trigger-label association, potentially reflecting overfitting. ScaleGMN can incorporate such patterns into its learned representation, making it well-suited to classify models as clean or infected.

The final ScaleGMN model, is similar to the one outlined in the original paper used for generalization prediction. It has an input and output dimension of 1 and 5 hidden layers of 128 dimensions.

2.2.2 Baseline

To benchmark the performance of our model, we use the Meta Neural Trojan Detection (MNTD) pipeline Xu et al. [2020] as a baseline. This model classifies trojaned neural networks using black-box access to the networks, where only the inputs and outputs to the network are used to identify the trojaned networks. This method deviates from ours in the manner that the ScaleGMN uses white-box access to the networks, where the model uses the network's weights and biases to identify trojaned neural networks.

2.3 Cleansing Trojaned Networks

Inspired by the INR editing task of the original paper, we applied ScaleGMN to restore the trojaned networks. To do this, we trained ScaleGMN on pairs of trojaned networks and their corresponding

healthy versions, adding the difference in weights of the healthy and the trojaned networks to the trojaned network, creating a new healthy model. For testing and application, a trojaned network input should output a healthy network. Then, to test whether cleansing was accomplished, we compare the resulting benign model's weights and biases to the original benign model's weights and biases using the MSE as the original paper's INR editing uses the same metric.

3 Results

3.1 Reproduction Results

The reproduction experiments show that the model from the original authors behaves in an expected manner. tables 1 and 2 show that for both the kendall-τ(KENDALL [1938]) metric for generalization prediction and the MSE metric for INR editing our results are similar to the original authors' results, supporting the author's claim on outperforming baselines in considered models and datasets.

Table 1: Reproduction of Generalization Prediction on CIFAR-10-GS SmallCNN Zoo with ReLU activation

Method	Kendall-τ (mean $\pm$ std)
ScaleGMN-B (Original)	0.9282 ± 0.003
ScaleGMN-B (Reproduction)	0.9287 ± 0.0014

Table 2: Reproduction of INR Editing. Dilating MNIST INRs. MSE between the reconstructed and ground-truth image

Method	MSE in 10^{-2} (mean $\pm$ std)
ScaleGMN-B (Original)	1.89 ± 0.00
ScaleGMN-B (Reproduction)	1.937 ± 0.029

3.2 Trojan Classification

Table 3 shows that our white-box method of Trojan detection outperforms the baseline in datasets using either trigger patterns. We notice that the MNTD does not perform with the same AUC as in their original experiments. We expect that the generation of the trojaned models has to do with this given that, while we inspired our trigger generation on the paper's modification trojaning method, we did not generate the trojaned models using jumbo learning as the MNTD paper did, but used the generated trigger to finetune the models for 2 epochs.

It is still very noticeable that the ScaleGMN greatly outperforms the baseline for the examined datasets while also showing that the bidirectional variant has a greater AUC than the forward ScaleGMN.

Table 3: Trojan classification: AUC Scores on subsets of SmallCNN Zoo, Trojaned & clean.

Method	CIFAR-10 White Square	CIFAR-10 Random Pattern
MNTD	75.5%	67.7%
ScaleGMN	99.6%	99.2%
ScaleGMN-B	**99.7%**	**99.6%**

3.3 Network Cleansing

Unfortunately, our method of cleansing by directly adjusting the parameters of the CNN did not provide useful results. During training, the metanetwork failed to find a function that could repair a trojaned network by steering its weights towards its healthy counterpart. As a result the loss of the model did not decrease and learning failed. While we were not successful, it is worth noting that our

approach of adding delta weights to the existing parameters is a very naïve method, and was very unlikely to achieve proper results in the first place. Ideally, instead of taking all CNN parameters as input and minimizing the distance to the healthy model, attention should be given to the emerging properties that can be found in trojaned model (Zheng et al. [2021]). This could result in using a subsection, or a different configuration of the CNN model parameters. Our loss function also adds the learned weight deltas to the existing weights, which is potentially a gross oversimplification. Weights and biases interact in specific ways in the model, and it is unlikely that a simple value adjustment to the weights would result in proper results. Also some type of regularization may be needed for the adjustment in weights to be learned efficiently by the metanetwork.

4 Discussion

In this work, we have introduced the ScaleGMN for detecting trojaned networks. The ScaleGMN has proved to be more accurate than state-of-the-art in detecting these networks. We have also applied the ScaleGMN to cleanse the trojaned networks, making these networks more robust to modification trojaning attacks and improving the accuracy of the previously trojaned models.

4.1 Limitations

As the original paper mentions, the ScaleGMN is only able to perform on fully connected FFNNs. While the model shows that it is able to classify trojaned models, these models are small and likely not used for varying tasks. We expect that more recent networks, such as transformers, will result in a lower accuracy, since the weight distributions are more complex for these networks.

4.2 Future Work

In future work we hope to be able to use a version of the ScaleGMN which is applicaple to more complex models, as Kalogeropoulos et al. [2024] stated that they wished to expand their model to these domains, which include networks that make use of skip connections and normalization layers. When they will eventually go beyond their limitations, we will be excited to apply this model to our experiments.

Using the bidirectional-ScaleGMN, we also wish to not only identify trojaned models, but also identify the trigger that was injected in the initial dataset and remove this trigger using INR editing, making the ScaleGMN even more accurate in identifying trojaned networks and making it possible to apply the ScaleGMN to make networks even more robust.

Lastly, as stated in section 3.3, our method of cleansing trojaned networks did not provide any useful results. We wish to keep experimenting with ways to cleanse these networks using methods defined in this section, so that eventually, the ScaleGMN can be applied to trojaned networks to cleanse them.

References

T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain, 2019. URL https://arxiv.org/abs/1708.06733.

I. Kalogeropoulos, G. Bouritsas, and Y. Panagakis. Scale equivariant graph metanetworks, 2024. URL https://arxiv.org/abs/2406.10685.

M. G. KENDALL. A new measure of rank correlation. *Biometrika*, 30(1-2):81–93, 1938.

A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. 2009.

B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 707–723, 2019. doi: 10.1109/SP.2019.00031.

X. Xu, Q. Wang, H. Li, N. Borisov, C. A. Gunter, and B. Li. Detecting ai trojans using meta neural analysis, 2020. URL https://arxiv.org/abs/1910.03137.

S. Zheng, Y. Zhang, H. Wagner, M. Goswami, and C. Chen. Topological detection of trojaned neural networks, 2021. URL https://arxiv.org/abs/2106.06469.

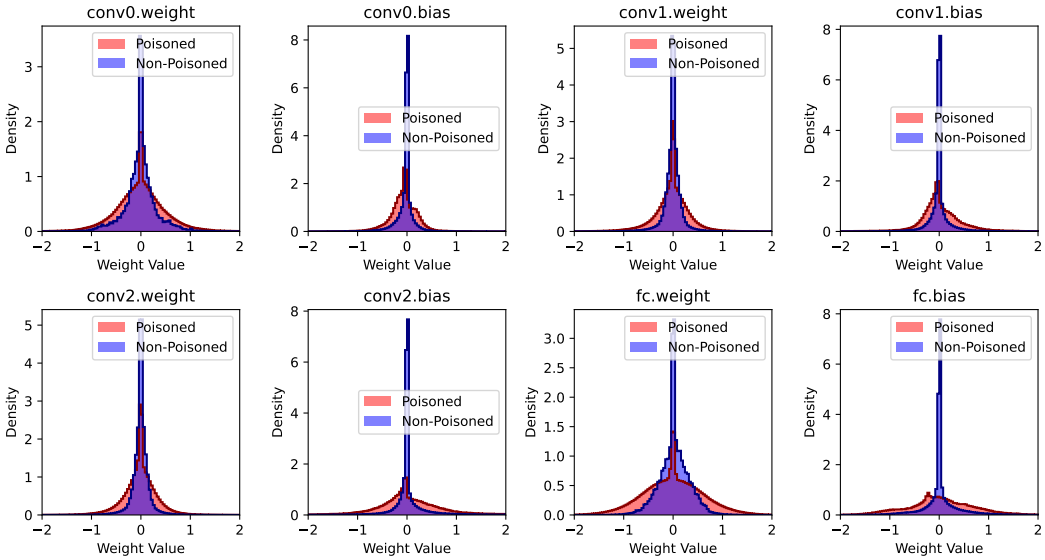


Figure 1: Weight and bias distributions per layer when using original clean models from CNNZoo

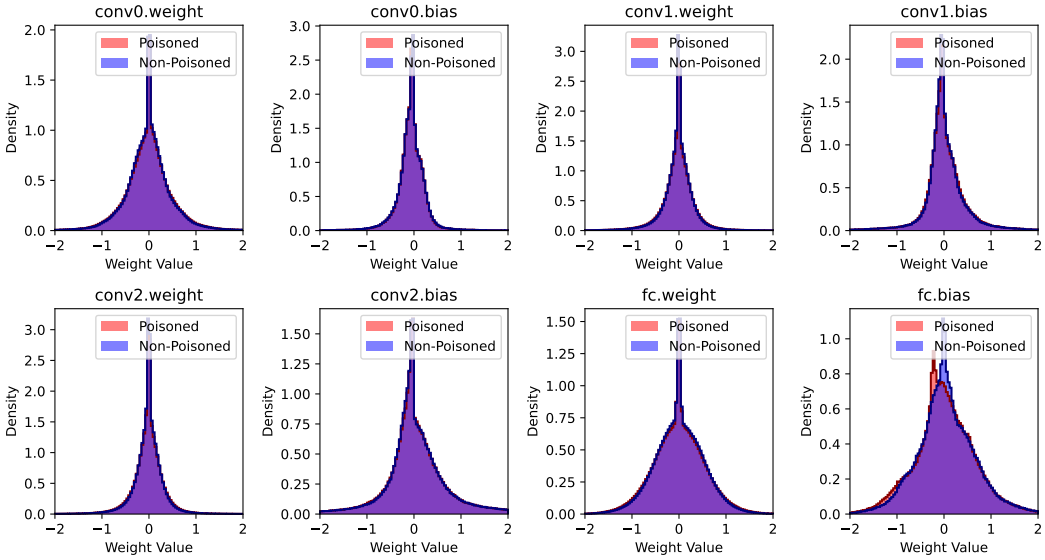


Figure 2: Weight and bias distributions per layer after fine-tuning the clean models for 2 epochs on the clean dataset.

A Dataset Distributions

Initially we chose not to fine-tune the clean CNNZoo models to save on time and resources. However, one peculiarity that arose from this decision was the fact that the distribution of weights and biases differed significantly between the Trojaned and clean models, as visible in Figure 1. We theorize that this is likely due to the fact that the different random number generation used by PyTorch to fine-tune the models lead to vastly different local minima, leading the weights to be less optimised. The result of this, was that ScaleGMN reached 100% AUC after only 1 epoch.

We managed to fix this by fine-tuning the clean models on clean data for 2 epochs, the same amount as for the Trojaned models. As visible in Figure 2, this largely created very similar distributions, except for the weights of the last linear layer. The result of this was a more expected AUC curve during training.

This is not to say the first method of using clean models that were not fine-tuned is inherently wrong; it is a plausible real-world scenario. What it does show, is how sensitive ScaleGMN is to differences in training setup.